CODEBASE REVIEW & PHASED ROADMAP

Autonomous AI Analyst: what to fix, in what order

The scaffolding is genuinely good — async job pipeline, content-addressed caching, clean module boundaries. The layers it wraps are where the problems live: a leaked API key, a leaking train/test split, mislabeled explanations, and a chatbot pointed at a file it cannot reach.

Reviewed 16 Aug 2026 **Scope** backend/ + frontend/, 24 source files **Stack** FastAPI · scikit-learn · XGBoost · SHAP · React 18 + Vite · Gemini

The verdict in four numbers

Severity ratings are about consequence, not effort — three of the four critical items are under an hour of work each.

BLOCKING	CORRECTNESS BUGS	TEST COVERAGE	SHIPPED ACCURACY
1	**6**	**0%**	**1.7%**
Live Gemini API key committed to a public GitHub repo	Silent — the app returns confident numbers that are wrong	No tests, no CI, no linting, unpinned dependencies	Real result in the committed model metadata — see below

Exhibit A: the model artifact checked into the repo

The one trained model in backend/models/metadata/ is not a toy — it's a 19,245-row healthcare dataset, and it documents four separate failures at once:

```
"selected_model": "DecisionTree",
"all_model_scores": { "DecisionTree": { "accuracy": 0.017, ... } },
"target": "Findings",
"failed_models": {
  "LogisticRegression": "Unable to allocate 1.67 GiB for an array with shape (15380, 14555)",
  "XGBoost": "Invalid classes inferred from unique values of `y`. Expected: [0 1 2 ... 14554],
              got ['35-year-old female enrollee has requested mental health outpatient...']"
}
```

Target detection grabbed the last column, which happened to be a free-text clinical narrative. Problem-type detection called 14,555 unique paragraphs a classification task. One-hot encoding exploded it to 14,555 columns and OOM'd logistic regression. XGBoost rejected the string labels. A decision tree "won" with 1.7% accuracy — and the UI presented it as the Optimized Model with no caveat.

Nothing in the pipeline flagged any of this. That's the theme of the review: the failure modes are silent, not loud.

Findings ledger

Ranked by consequence. The right-hand column is the phase that closes it.

FINDING	SEV	LOCATION	WHY IT MATTERS	PH
Live API key committed	CRIT	backend/.env	A 41-char GEMINI_API_KEY is tracked in git and pushed to a public remote. .gitignore lists .env, but the file was added before that rule existed, so git keeps tracking it.	0
Train/test leakage	CRIT	ml/preprocess.py:55 ml/train.py:41	The imputer, scaler, and one-hot encoder are fit_transform-ed on the entire dataset, and only then is the data split. Test-set statistics bleed into training. Every reported score is optimistic by an unknown margin.	1
SHAP names misaligned	CRIT	ml/explain.py:20	Importances are computed over the post-one-hot matrix but labeled with the raw column list. With any categorical column present, index i of the matrix is not column i of the dataframe — so "Key Feature Influencers" shows real numbers attached to the wrong features.	1
Chat reads an unreachable path	CRIT	routes/chat.py:85	The prompt tells Gemini to pd.read_csv() a local Windows path, but code execution runs in Google's remote sandbox with no access to your disk. The stored path is also stale (…\Desktop\AI analyst\…, a folder since renamed). The headline feature answers from summary stats or invention, not the data.	1
Naive target & task detection	CRIT	ml/preprocess.py:16 ml/train.py:15	Target = last column unless literally named "target"/"label", with no user override. Task type = "≤20 unique values or ratio < 0.1 → classification", which misreads free text, IDs, and low-cardinality regression targets. This is what produced Exhibit A.	1
No label encoding for XGBoost	HIGH	ml/train.py:26	String class labels are passed straight to XGBClassifier, which requires 0…n−1 integers. It always fails on string targets — swallowed into failed_models and never surfaced prominently.	1
Cache key ignores training mode	HIGH	routes/upload.py:91	The cache is keyed on dataset hash alone. Re-upload the same CSV in Ensemble mode after Auto and you silently get the Auto result back — the mode selector appears to do nothing.	1
Unvalidated hash → pickle load	HIGH	routes/predict.py:14-16	dataset_hash is interpolated into a file path with no format check, then handed to joblib.load. Path traversal into arbitrary pickle deserialization is remote code execution. One regex fixes it.	0
CORS wildcard + credentials	HIGH	backend/main.py:14	allow_origins=["*"] with allow_credentials=True — an invalid combination browsers reject, and wide open the moment the API leaves localhost. No auth or rate limiting on any route, including the paid LLM endpoint.	0
Unbounded upload	HIGH	routes/upload.py:239	await file.read() pulls the whole file into RAM with no size	0

FINDING	SEV	LOCATION	WHY IT MATTERS	PH
into memory			cap, type check, or row limit. A 2 GB upload takes the server down.	
Job state is a module dict	HIGH	routes/upload.py:16	UPLOAD_JOBS lives in process memory: lost on restart, grows forever, and breaks entirely under more than one uvicorn worker.	2
Charts computed, never drawn	MED	routes/upload.py:126 components/Dashboard.jsx	Histogram and bar payloads are built, stored, and returned — and nothing renders them. /api/insights has no client caller at all. Two backend features exist with no way in.	3
ASCII-only sanitization	MED	utils/helpers.py:61	Every character outside 32–126 is stripped, then columns >60% empty are rejected. Any non-English dataset is mangled or refused outright.	3
Silent row-dropping	MED	utils/helpers.py:36	on_bad_lines="skip" discards malformed rows without counting them. The warning text is generic and never says how many rows vanished.	3
response.text may be None	MED	routes/chat.py:100	With code execution enabled the reply can carry executable-code and result parts and no plain text, making .strip() raise — surfaced to the user as a raw exception string.	1
Frontend not deployable	MED	services/api.js:1	API base is hardcoded to localhost:8000 with no env var, and the built dist/ bundle is committed to git.	2
Polling loop never cancels	MED	pages/UploadPage.jsx:34	A while loop polls every second with no unmount cleanup, no timeout, and no abort — it keeps running and calling setState after the component is gone.	3
No tests, CI, or pinning	MED	repo-wide	Zero test files. requirements.txt is 12 unpinned names, so a scikit-learn or google-genai release can break the build silently. No Dockerfile, no lockfile discipline, no logging.	2

The phases

Sequenced so that each phase is safe to ship on its own. Phases 0 and 1 are remediation and should not be reordered — everything after is genuine capability. Estimates assume one developer.

PHASE 0 · 2–3 HOURS · DO THIS TODAY

Stop the bleeding

A live credential is public and two endpoints accept unvalidated input. Nothing else in this document matters until these are closed.

- **Rotate the Gemini key first.** Revoke it in Google AI Studio and issue a new one. Rotation is what actually protects you — history rewriting is cleanup, not remediation, because the old key may already be scraped.
- **Purge it from git.** `git rm --cached backend/.env`, commit, then rewrite history with `git filter-repo` or BFG and force-push. Also untrack `frontend/dist/` and add both to `.gitignore`. Consider whether the committed 19k-row healthcare dataset snapshot belongs in a public repo either.
- **Validate dataset_hash** against `^[a-f0-9]{64}$` in both predict.py and chat.py before it ever touches a path, and resolve the result under a known root. This closes the traversal-to-pickle path.
- **Cap uploads** — a max byte size, an extension/content-type check, and a row ceiling, enforced by streaming to a temp file rather than read()-ing into memory.
- **Tighten CORS** to an explicit origin list from an env var, and drop allow_credentials until there is an auth story that needs it.
- **Add a secret scanner** — pre-commit with detect-secrets or gitleaks — so this cannot recur.

EXIT CRITERIA

Old key revoked and absent from all history; both hash-taking routes reject a non-hex input with 400; a 1 GB upload is refused without touching RAM.

PHASE 1 · 3–5 DAYS · TRUST THE NUMBERS

Make the output honest

Right now the app is confidently wrong: leaked scores, mislabeled explanations, and an analyst chatbot that cannot see the data it claims to compute on. Fix correctness before adding anything.

- **Split before you fit.** Move train_test_split ahead of preprocessing and wrap the ColumnTransformer plus estimator in a single sklearn Pipeline, fit on train only. This kills the leakage and makes serving a single artifact instead of two, which also simplifies predict.py.
- **Fix feature attribution.** Pass `preprocessor.get_feature_names_out()` into explain_model so SHAP values carry their true names, and aggregate one-hot columns back to the parent feature for display. Stop swallowing the exception silently — log when SHAP falls back to feature_importances_.

- **Rebuild target and task detection.** Add an optional target_column parameter to the upload API with a column picker in the UI, defaulting to the current heuristic. Reject targets that are high-cardinality free text with a clear message ("'Findings' has 14,555 unique values across 19,245 rows — this looks like free text, not a label"), and add a cardinality ceiling on one-hot encoding so a text column can never explode into 14k features.
- **Encode labels properly.** Wrap classification targets in a LabelEncoder stored alongside the model, inverse-transforming at predict time. XGBoost then works on string labels instead of always failing.
- **Key the cache on the full configuration** — dataset hash + mode + manual model + target + a pipeline version string. A code change to preprocessing should invalidate stale artifacts automatically.
- **Decide what the chatbot actually is.** Two honest options: (a) upload the CSV to the Gemini Files API and reference it, so remote code execution genuinely has the data; or (b) drop Gemini's sandbox and run a restricted pandas executor server-side against the local snapshot. Option (a) is faster to ship; option (b) keeps the data on your machine. Either way, stop instructing the model to read a local path, and handle a None response.text by walking the response parts.
- **Surface failures in the UI.** failed_models and quality warnings already reach the frontend and get buried in a bullet list. Give a run that scored 1.7% a visible health verdict instead of the label "Optimized Model".

EXIT CRITERIA
Scores computed by a leak-free pipeline; importance labels match real column names on a categorical dataset; the chatbot's arithmetic is reproducible against the CSV by hand; re-uploading in a different mode retrains.

PHASE 2 · 4–6 DAYS · FOUNDATION

Make it an engineered project

Phase 1's fixes are only durable if something catches their regression. This is the phase that turns a working prototype into a maintainable codebase.

- **Tests where the bugs were.** pytest with fixture CSVs covering: leakage (preprocessor is fit only on train rows), feature-name alignment through one-hot, target detection on adversarial columns, cache-key behavior across modes, and the path-traversal rejection. Add Vitest plus Testing Library for the upload → poll → render flow.
- **Configuration and logging.** Replace scattered os.getenv with a pydantic-settings object; add structured logging with a request/job id so a failed pipeline is diagnosable from logs instead of a single string on a dict.
- **Persist job and dataset state.** Move UPLOAD_JOBS to SQLite via SQLModel (Postgres later) so jobs survive restarts, support multiple workers, and give you a dataset registry for the history UI in Phase 3.
- **Pin and containerize.** Compile requirements.txt with pip-tools or move to uv; add a backend Dockerfile, a frontend build stage, and docker-compose for one-command startup.
- **CI on every push** — ruff + black, pytest, npm build, and a secret scan. Fail the build on any of them.
- **Make the frontend deployable.** VITE_API_BASE with a dev proxy, an error boundary around the dashboard, and remove the committed bundle.

- **Small cleanups:** replace deprecated datetime.utcnow(), and give the pipeline real exception types instead of raising HTTPException inside a background task where its status code is meaningless.

EXIT CRITERIA
docker compose up brings up both services; CI is green; killing the backend mid-job and restarting still shows correct job state.

PHASE 3 · 1 WEEK · CLOSE THE LOOP

Ship the features already half-built

The backend computes chart payloads, summary stats, and a quality report that the UI never shows, and exposes an insights endpoint nothing calls. This phase is unusually high value-per-hour because the hard half is done.

- **Render the charts.** Wire result.charts into real histograms and bar charts, plus a correlation heatmap and a confusion matrix (classification) or residual plot (regression). Reach for a charting library rather than hand-rolling divs — the current feature-importance bars are CSS width percentages with no axis or scale.
- **Build a dataset workspace.** Backed by the Phase 2 registry: list past datasets, reopen a run, compare model scores across runs, delete a dataset and its artifacts. Today every result vanishes on refresh.
- **Show data quality up front.** A per-column profile — type, missing %, cardinality, distribution sparkline — before training starts, with the target picker built into that same screen.
- **Finish the prediction flow.** Return class probabilities and confidence, render results as a table rather than a raw JSON dump, offer CSV download of predictions, and support pasting a single row instead of requiring a file upload.
- **Honest parsing feedback.** Count skipped malformed rows and report the number; replace ASCII-stripping with proper Unicode normalization so non-English datasets survive.
- **Fix the polling loop** with an AbortController, unmount cleanup, backoff, and a timeout — or switch to server-sent events, since the backend already tracks incremental progress.

EXIT CRITERIA
Every field the backend returns is visible somewhere in the UI; a returning user can find yesterday's dataset; a Japanese-language CSV trains without mangling.

PHASE 4 · 1–2 WEEKS · DEPTH

Make the ML defensible

A single 80/20 split with fixed hyperparameters and three metrics is a demo. This is what separates it from an AutoML tool someone would actually trust.

- **Cross-validation with stratification** instead of one split, reporting mean ± std so a lucky split can't win. Add a held-out final test set the tuner never sees.

- **Real metrics.** ROC-AUC, F1, and per-class breakdowns for classification; MAE, R^2, and MAPE for regression. Selecting on raw accuracy is actively misleading on imbalanced data — the current _select_best_model would pick a majority-class predictor.
- **Handle class imbalance** — class weights, optional SMOTE, and a warning when the majority class exceeds a threshold.
- **Hyperparameter search** via Optuna or randomized search with a time budget the user sets, rather than hardcoded n_estimators=100.
- **Better ensembling.** Replace the mode/mean vote with sklearn's VotingClassifier / StackingClassifier weighted by validation performance, and stop including models that scored near chance.
- **Broaden the registry** — LightGBM, CatBoost, gradient boosting, ridge/lasso — and add categorical-native handling so high-cardinality columns use target or ordinal encoding instead of one-hot.
- **Model cards.** Persist the training configuration, data profile, metrics, and library versions per run, so a result from three months ago is still interpretable.

EXIT CRITERIA
Reported scores are CV means with variance; an imbalanced dataset produces a warning and a sensible model rather than a high-accuracy constant predictor.

PHASE 5 · 1–2 WEEKS · THE DIFFERENTIATOR

An analyst that actually analyzes

This is the project's real ambition and the part most worth investing in — but only on top of Phase 1's honest foundation. An agent reasoning over leaked metrics and mislabeled features produces fluent nonsense.

- **Server-side sandboxed execution.** A restricted Python worker — subprocess with resource limits, or a container — that runs generated pandas code against the actual snapshot, returns stdout, dataframes, and rendered plots, and enforces a timeout. This is what the README already promises.
- **Tool-calling over free-form code.** Define explicit tools (describe_column, correlate, filter_rows, plot, run_model) so common questions take a deterministic path and only genuinely novel ones fall through to arbitrary code. More reliable and much cheaper.
- **Render what the agent produces** — code blocks with syntax highlighting, result tables, and inline images — instead of collapsing everything to markdown text.
- **Persist conversations** per dataset, with server-side history rather than trusting whatever the client posts, plus token budgeting so long sessions don't silently truncate context.
- **Abstract the provider** behind an interface so the LLM is swappable and testable with a fake — currently chat.py hardwires the Gemini client. Note the README and UI both still advertise "Gemini 1.5 Flash"; pin and display the model you actually configure.
- **Guard the endpoint** — per-session rate limits, a token ceiling, and a graceful degraded mode when the key is missing or quota is exhausted.
- **Autonomous report generation:** one button that runs a full EDA sweep and emits a shareable report. This is the natural product headline once the machinery underneath is trustworthy.

EXIT CRITERIA

A question like "which two features correlate most strongly, and does that hold within each segment?" is answered by executed code whose output you can verify by hand.

PHASE 6 · 1–2 WEEKS · ONLY IF GOING LIVE

Multi-user and production

Everything so far assumes a single trusted operator on localhost. Skip this phase entirely if that stays true — it is real work and only pays off with real users.

- **Authentication and per-user isolation** so datasets, models, and conversations are scoped to an owner. Note that content-addressed caching means two users uploading the same file share artifacts — decide deliberately whether that is a feature or a data leak.
- **A real job queue** — Celery or RQ with Redis — replacing FastAPI BackgroundTasks, which shares the web process and dies with it.
- **Object storage** (S3 or equivalent) for models and snapshots instead of the local filesystem, so the app can scale past one machine.
- **Observability** — health and readiness probes, Prometheus metrics, Sentry, and LLM cost tracking per request.
- **Data lifecycle** — retention policy, deletion endpoint, and a documented answer to "where does my uploaded data go", which matters more than usual given healthcare-shaped datasets are already in scope.
- **Replace pickle** for model serialization, or at minimum sign artifacts — joblib.load on anything a user can influence is a permanent RCE surface.

EXIT CRITERIA

Two users cannot see each other's data; the API survives a worker restart mid-training; every stored artifact has an owner and an expiry.

In closing

If you only do one thing

Rotate the Gemini key. It is live, public, and billable to you right now. Everything else can wait a day; that cannot.

If you only do one week

Phase 0 plus Phase 1. That is the difference between software that produces wrong answers confidently and software whose answers you can stand behind.

Where the leverage is

Phase 3. The backend already computes charts, stats, and quality reports that nothing displays — the highest ratio of visible improvement to work in the whole plan.

What's genuinely good

Module boundaries are clean, the async job pattern with progress logging is well built, content-addressed caching is a smart idea, and CSV parsing is more defensive than most. The architecture holds; the layers need work.